

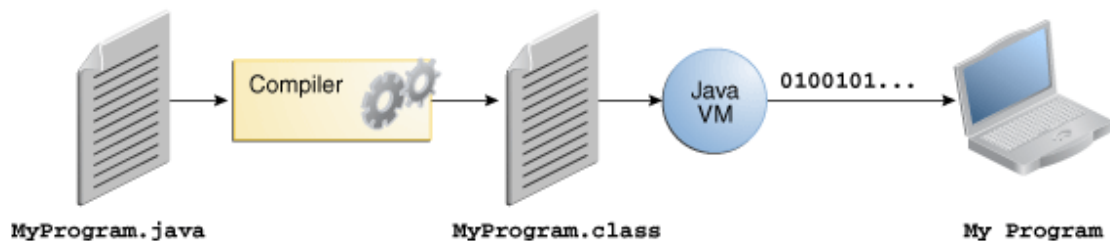
ACERCA DE LA TECNOLOGÍA JAVA

La tecnología Java es tanto un lenguaje de programación como una plataforma.

El lenguaje de programación Java es un lenguaje de alto nivel que se puede caracterizar por todas las siguientes palabras de moda:

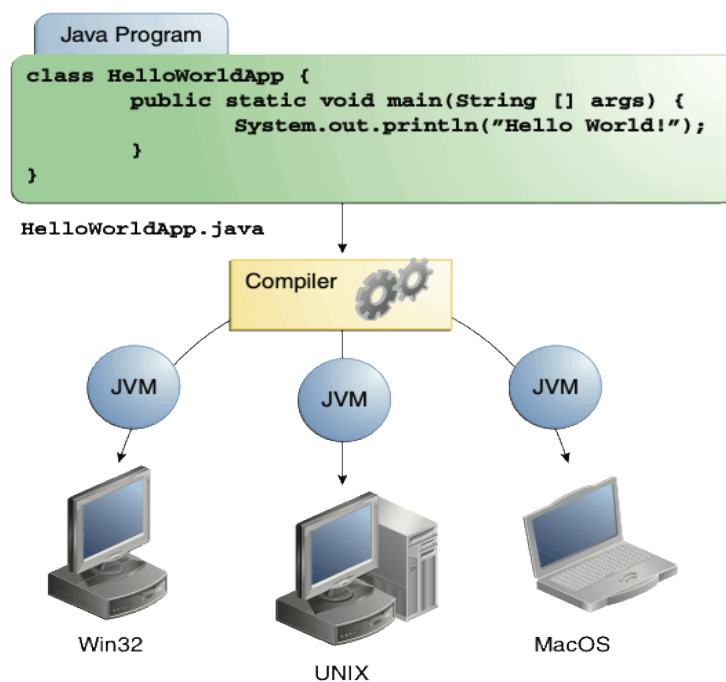
- Simple
- Orientado a objetos
- Abierto
- multiproceso
- Dinámica
- Arquitectura neutra
- Portátil
- Alto rendimiento
- Robusto
- Seguro

En el lenguaje de programación Java, todo el código fuente se escribe primero en archivos de texto sin formato que terminan con la **.java** extensión. Esos archivos fuente luego son compilados en **.class** archivos por el **javac** compilador. Un **.class** archivo no contiene código nativo de su procesador; en su lugar, contiene **bytecodes**, el lenguaje de máquina de Java Virtual Machine 1 (Java VM). La java herramienta de inicio luego ejecuta su aplicación con una instancia de la **máquina virtual de Java**.



Una visión general del proceso de desarrollo de software.

A través de Java VM, la misma aplicación es capaz de ejecutarse en múltiples plataformas.



LA PLATAFORMA JAVA

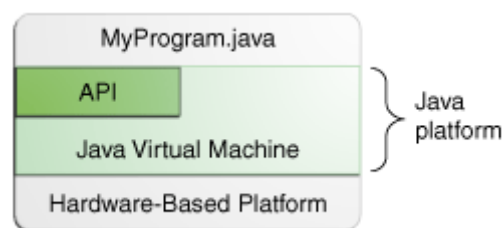
Una plataforma es el entorno de hardware o software en el que se ejecuta un programa. Ya hemos mencionado algunas de las plataformas más populares como Microsoft Windows, Linux, Solaris OS y Mac OS. La mayoría de las plataformas se pueden describir como una combinación del sistema operativo y el hardware subyacente. La plataforma Java se diferencia de la mayoría de las otras plataformas en que es una plataforma solo de software que se ejecuta sobre otras plataformas basadas en hardware.

La plataforma Java tiene dos componentes:

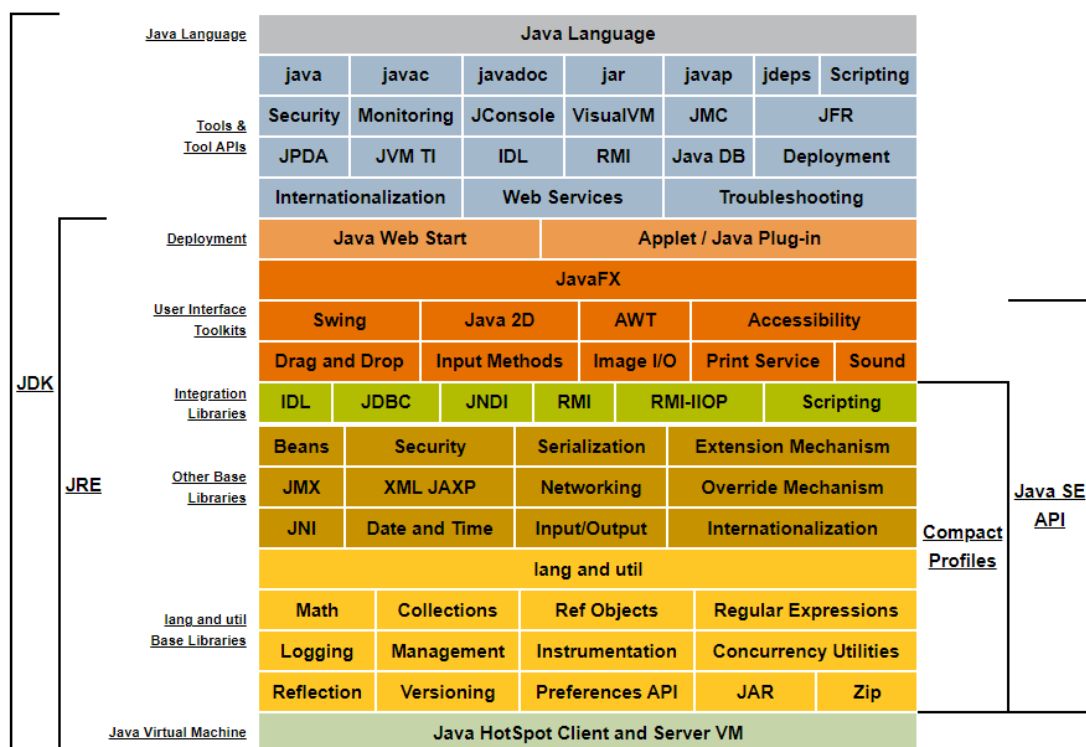
- La máquina virtual de Java
- La interfaz de programación de aplicaciones Java (API)

Ya conoce la máquina virtual de Java; es la base de la plataforma Java y se adapta a varias plataformas basadas en hardware.

La API es una gran colección de componentes de software listos para usar que brindan muchas capacidades útiles. Se agrupa en bibliotecas de clases e interfaces relacionadas; estas bibliotecas se conocen como paquetes.



La API y la máquina virtual de Java aíslan el programa del hardware subyacente.



Los términos "Máquina Virtual Java" y "JVM" significan una Máquina Virtual para la plataforma Java.

ELEMENTOS DEL LENGUAJE JAVA

Comentarios:

Tres tipos de comentarios:

// De una sola línea

**/* De una ó
más líneas
*/**

/ De documentación, de una o varias líneas.
Justo antes del elemento que documenta
(clase, método, etcétera)
*/**

- Los comentarios de documentación son utilizados por javadoc para generar documentación en formato HTML. Pueden llevar etiquetas.

Comentarios: etiquetas para javadoc

Etiquetas para referencias:

@see <otra clase>

Etiquetas de documentación de clases:

**@version <información sobre la versión>
@author <nombre autor>**

Etiquetas de documentación de métodos:

**@param <nombre argumento><descripción>
@return <descripción>
@exception <excepción>**

Ejemplo:

```
// Hola.java

/** Primera clase de prueba
 * @author Luis Hernández
 * @version 1.0
 * @see Adios
 */
```

Identificadores:

En Java, un identificador comienza con una letra, un subrayado (_) o un símbolo de dólar (\$). Los siguientes caracteres pueden ser letras o dígitos.

Java distingue entre mayúsculas y minúsculas
--

Convenios:

- Nombre de clase: empieza por mayúscula.
- Nombre de método o atributo: empieza por minúscula (quizá con un subrayado delante para los atributos).
- Las constantes: todo en mayúsculas.
- Identificadores formados por varias palabras: comienzo de cada nueva palabra en mayúsculas (calcularSuma, objetoPersona).

abstract	continue	for	new	switch
boolean	default	goto	null	synchronized
break	do	if	package	this
byte	double	implements	private	threadsafe
byvalue	else	import	protected	throw
case	extends	instanceof	public	transient
catch	false	int	return	true
char	final	interface	short	try
class	finally	long	static	void
const	float	native	super	while

Separadores:

- **()** Para delimitar listas de parámetros o de argumentos.
Para modificar la precedencia en una expresión.
Para delimitar condiciones.
Para realizar conversiones de tipo (moldes o casting).
- **{ }** Para definir bloques de código.
Para delimitar listas de valores iniciales de los vectores (arrays).
- **[]** Para declarar vectores y referenciar elementos de los mismos.
- **;** Para terminar instrucciones.
- **,** Para separar identificadores en declaraciones.
Para encadenar expresiones (operador coma).
- **.** Para acceder a atributos de objetos u objetos globales de clases.
Para pasar mensajes a objetos.
Para acceder a un subpaquete de un paquete.

Método main

Si echamos un vistazo a la siguiente línea **public static void main(String args[])** podemos sacar algunas conclusiones que es conveniente ir aclarando y memorizando. Resumimos todas ellas en una pequeña tabla para que le sea más fácil la comprensión de cada uno de los elementos que componen esta línea. (ver tabla).

ELEMENTO A ESTUDIO	INFORMACIÓN APORTADA
Main	El intérprete de Java busca un método para ejecutar en primer lugar, éste viene dado por main () y con ella se especifica la primera función que se debe ejecutar.
Public	Al ser público se indica que el método main () podría ser llamado por cualquiera, incluido el propio intérprete Java.
static	Nos indica que el método main () se refiere a la propia clase ejercicio1 y no a ninguna instancia de la clase.
Void	Indica que main() no devuelve nada. Esto es importante porque Java realiza una comprobación de tipos.
args[]	Es la declaración de un array de String, estos son los argumentos escritos tras el nombre de la clase en la línea de comandos.

Tipos de datos en JAVA

El lenguaje Java define 8 tipos de datos primitivos:

byte	short	int	long
float	double	char	boolean

Datos de tipo numérico

- Números enteros byte, short, int, long
- Números en coma flotante float, double

Datos de tipo carácter

char

Datos de tipo booleano

boolean

Tipo de dato	Espacio en memoria	Valor mínimo	Valor Máximo
byte	8 bits	-128	127
short	16 bits	-32768	32767
int	32 bits	-2147483648	2147483647
long	64 bits	-9223372036854775808	9223372036854775807

Tipo de dato	Espacio en memoria	Mínimo (valor absoluto)	Máximo (valor absoluto)	Dígitos significativos
float	32 bits	1.4×10^{-45}	3.4×10^{38}	6
double	64 bits	4.9×10^{-324}	1.8×10^{308}	15

Variables: declaración

Tipo_variable nombreDeLaVariable;

Tipo_variable nombreDeLaVariable = expresión; // inicialización de la variable

Tipo_variable nombreDeLaVariable1, nombreDeLaVariable2,...;

Tipo_variable nombreDeLaVariable1 = exp1, nombreDeLaVariable2 = exp2, ...;

Ejemplos:

```
long suma = 234;
int edad, valor, num1, result;
int altura = 23, ancho = 45;
double resultado = 0.0;
float area = 23 * 45;
boolean encontrado = false;
char nl = '\n';
```

Constantes de clase

- Accesibles en toda la clase.
- Se declaran en la sección de declaraciones globales de la clase.
- Se declaran como **static** (de clase) y **final** (inmodificables), asignándoles el valor en la declaración.

```
public class Pruebas {  
    static final double PI = 3.14159265;  
    public static void main (String[] args) {  
        System.out.println(PI);  
    }  
}
```

```
String mensaje="Hola mundo";
```

Operadores:

Operador de asignación =

variable = expresión;

Operadores aritméticos

Unarios: + - ++ (incremento) -- (decremento)

x = y++; // Se asigna y a x; luego se incrementa y

x = ++y; // Se incrementa y; luego se asigna y a x

Binarios: + - * / % (módulo)

Ternario: ? (x < y) ? x : y (si x < y, entonces x, si no y)

Abreviaturas

x += y; equivale a x = x + y;

x -= y; equivale a x = x - y;

x *= y; equivale a x = x * y;

x /= y; equivale a x = x / y;

x %= y; equivale a x = x % y;

Operadores relacionales == != < <= > >=

Operadores lógicos (booleanos) Evaluación perezosa

! Negación (NOT)

&& Conjunción (AND)

|| Disyunción (OR)

^ O exclusiva (XOR)

Prioridad de operadores:

Mayor

++ -- (prefijos)

!

* / %

+ -

< <= > >=

== !=

&

^

|

&&

||

Menor = += -= *= /= %=

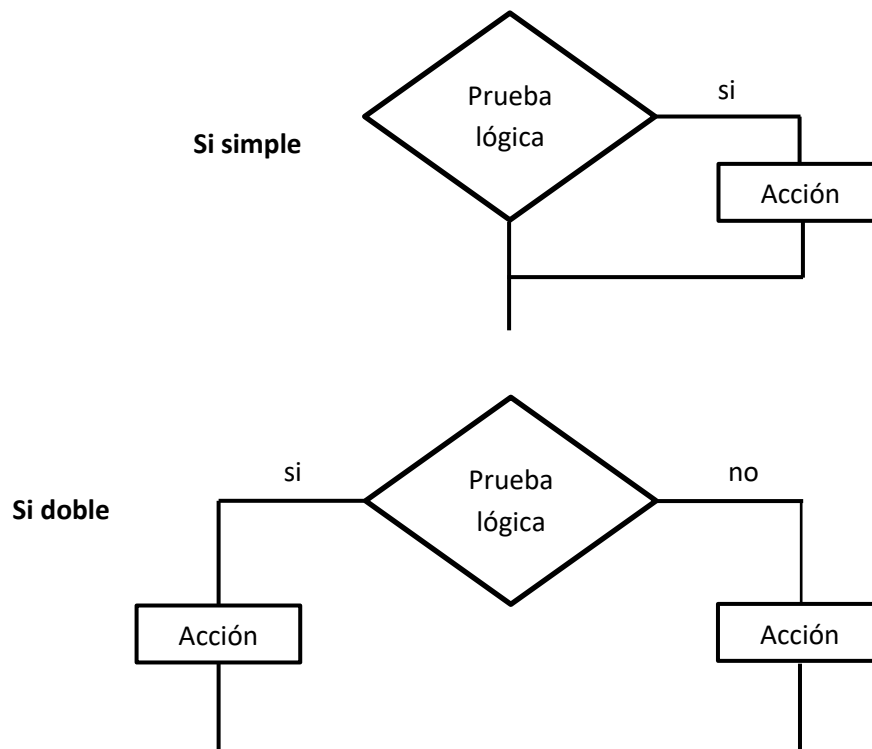
ESTRUCTURAS CONDICIONALES (Instrucciones de control del flujo)

Selección condicional (if): posibilita elegir qué parte de un programa se ejecutará si cierta condición se cumple.

La sintaxis básica de la sentencia **si** es:

```
if ( condición )  
{  
    Instrucción o bloque IF...  
}else{  
    Instrucción o bloque ELSE...  
}
```

La cláusula “**ELSE**” y las sentencias correspondientes son opcionales: solo se usan cuando el programa necesita tomar acciones si la condición es falsa.



IF-ELSE anidados: cada ELSE se asocia al IF anterior más cercano.

```
if(n == 1 ) {  
    // Bloque 1  
}  
else if(n == 2) {  
    // Bloque 2  
}  
else {  
    // Si todo falla, Bloque 3  
}
```

Ejercicios:

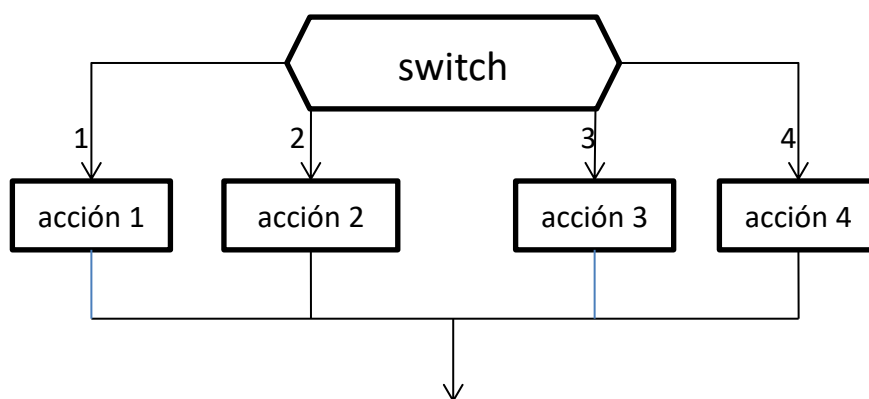
1. Diseñar el programa un programa que pida por teclado dos números enteros y muestre su suma, resta, multiplicación, división y el resto (módulo) de la división.
2. Diseñar un programa que permita registrar la venta de un producto, si el precio es mayor a \$50.000 realizar un descuento del 4% y si no 6%.
3. Diseñar un algoritmo que permita registrar las tres notas de un estudiante y obtener la nota final. Si la nota es mayor a 3 mostrar un mensaje "APROBADO" y si no "REPROBADO".

Selección múltiple (switch): Es funcionalmente equivalente a la forma extendida de la sentencia `if`, pero aporta mayor legibilidad cuando las condiciones son varias o complejas.

La sintaxis de la sentencia `switch` es:

```
switch(expresión) {  
  case valor1:  
    instrucciones1  
    [break;]  
  case valor2:  
    instrucciones2  
    [break;]  
  ...  
  [ default:  
    por-defecto ]  
}
```

```
switch(mes) {  
  case 1:  
    S.o.p("Enero");  
    break;  
  case 12:  
    S.o.p("Diciembre");  
    break;  
  default:  
    S.o.p("Valor no valido");  
}
```



El programa verificará el valor de verdad de la primera condición. Si se cumple, ejecutará todas las acciones definidas para ese caso en particular y luego irá a ejecutar la sentencia que siga a la llave que cierra el cuerpo de la sentencia `switch`. Si no se cumple, pasará a verificar la siguiente condición y así sucesivamente. Si ninguna condición fuese verdadera y existiese una cláusula "sino", se ejecutarán las acciones definidas bajo esta cláusula. La cláusula "default" es opcional. Si está presente debe ser la última.

Estructuras repetitivas (bucles)

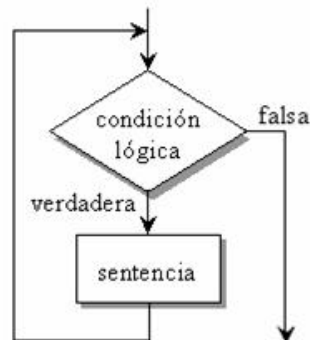
Ciclo mientras (while): Esta construcción posibilita ejecutar repetidamente un conjunto de Sentencias mientras se cumpla cierta condición definida por el programador. Debemos notar que la condición va siempre entre paréntesis.

El ciclo mientras tiene una peculiaridad: las sentencias contenidas en el ciclo puede que nunca se ejecuten. Esto ocurrirá cuando la condición del ciclo sea falsa incluso antes de entrar al ciclo.

La sintaxis del ciclo mientras es:

```
while ( condición )  
{  
    sentencias...  
}
```

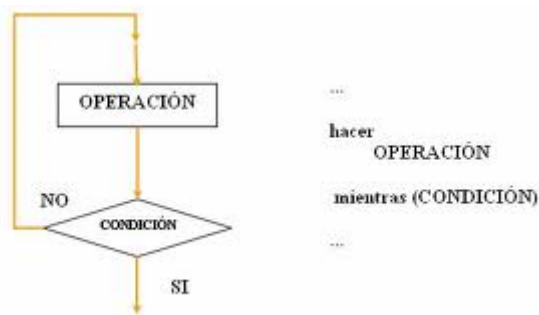
Diagrama de Flujo:



Ciclo repetir...hasta (DO ..While): A diferencia del ciclo mientras, el ciclo DO ..While ejecuta al menos una vez las sentencias definidas en el cuerpo del ciclo, pues la verificación para continuar o no en el ciclo se realiza al final, como puede apreciarse en la sintaxis:

```
Do{  
    sentencias...  
}while ( condicion_de_fin )
```

Debemos observar que la condición de fin va entre paréntesis.

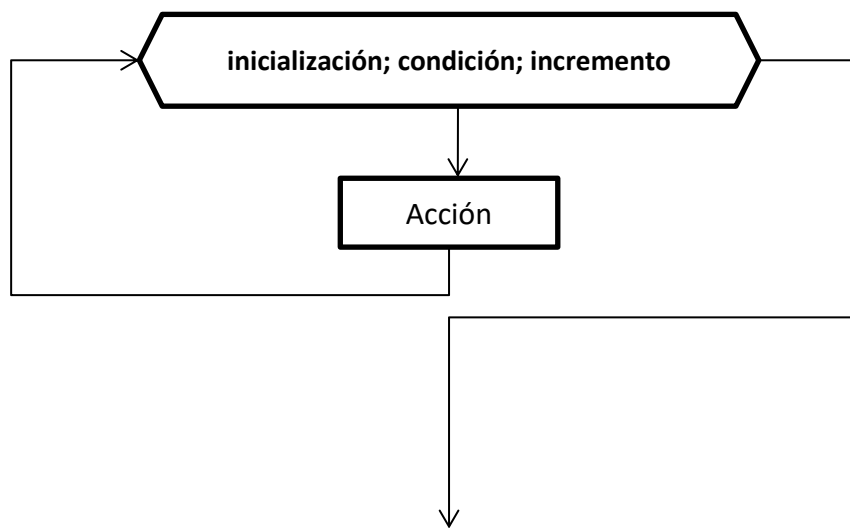


Si bien esta forma de ciclo es utilizado con menor frecuencia que el ciclo mientras(while), hay situaciones en las que con él el algoritmo resultante es más sencillo y directo. A continuación un ejemplo:

Ciclo desde(for): Utilizamos el ciclo **FOR** cuando conocemos previamente el número de veces que deseamos ejecutar un conjunto de instrucciones.

La sintaxis es:

```
for(inicialización;condición;incremento) {  
  {  
    secuencia-de-instrucciones...  
  }  
}
```



Al igual que el ciclo mientras, el conjunto de sentencias que forman parte de un ciclo desde puede que nunca se ejecute.

Esto ocurrirá si:

- El valor inicial es superior al valor final y el incremento es positivo, ó
- El valor inicial es inferior al valor final y el incremento es negativo.

Más detalles acerca del ciclo desde

Cuando usamos el ciclo desde debemos tener presente algunos detalles:

- El valor de inicio, el valor de fin y el incremento deben ser expresiones numéricas. El primero (inicio) y el último (incremento) son siempre automáticamente truncados a enteros.
- El valor del incremento puede ser negativo (por ejemplo, para ir de 100 a 1).
- El ciclo puede realizarse de 0 a n veces.
- La variable de control debe ser numérica, pudiendo ser un elemento de un vector o matriz.
- Alterar el valor de inicio, el valor de fin o el incremento no tiene ningún efecto sobre la cantidad de veces que se ejecutará el ciclo (esto se calcula justo antes de iniciar el ciclo).

- Si se desea salir del ciclo en forma anticipada, puede asignarse a la variable de control el valor de fin.

METODOS

Sentencia de llamada a Métodos: Un método puede ser ejecutado escribiendo su nombre, seguido de los parámetros reales que ella necesite. Podemos invocar la ejecución de subrutinas que hemos escrito en nuestro programa u otras que estén predefinidas en JAVA..

La sintaxis de llamada a método es:

Ejemplo con parámetros:

```
nombre_metodo (parametro1, parametro2, ...)
```

Ejemplo sin parámetros:

```
nombre_metodo ()
```

La cantidad de parámetros y el tipo de dato de cada parámetro depende de la definición de la subrutina.

Los métodos de las clases son las unidades funcionales de Java. Son similares a las funciones de otros lenguajes.

```
[modificadores] tipo nombre (parámetros) {
```

```
//cuerpo del método.
```

```
}
```

- Modificadores: indican cómo (dónde) se puede invocar el método. De momento, simplemente **static**.
- Tipo: tipo de dato que devuelve el método (**void** sin nada).
- Parámetros: declaraciones separadas por comas.

Ejemplo

El siguiente programa imprime la suma de los primeros n números enteros.

```
public class SumaMetodo {  
    public static void main(String[] args) {  
        int n1=3;  
        int n2=4;  
  
        //Llamamos al método y enviarnos dos valores  
        System.out.println("Respuesta: "+ sumaValores(n1, n2));  
    }  
  
    //metodo que recibe 2 valores y realiza la suma  
    static int sumaValores(int num1, int num2){  
        int suma=num1+num2;  
        return suma; //Se retorna el resultado de la suma  
    }  
}
```

Tipo de valor retornado

Parámetros que se pasarán como argumentos

Valor devuelto por el método

Sentencia retorna: Finaliza la ejecución de una subrutina que retorna un valor y debe ser la última sentencia, por lo que solo puede aparecer una vez en cada método.

La sintaxis es:

retorna expresión ;

Métodos: sobrecarga

Los métodos se pueden sobrecargar: varios métodos con igual nombre pero diferente signatura.

```
public class X {  
    public static void main(String[] args){  
        ...  
    }  
    static int max(int num1, int num2) { ... }  
    static long max(long num1, long num2) { ... }  
    static float max(float num1, float num2) { ... }  
    static double max(double num1, double num2) { ... }  
}
```

Bibliografía:

Documentación JAVA: <https://docs.oracle.com/javase/tutorial/getStarted/intro/cando.html>

Programación: <https://www.fdi.ucm.es/profesor/luis/fp/FP.pdf>

Fundamentos de Programación:

<http://assets.mheducation.es/bcv/guide/capitulo/844814645X.pdf>

- Sonia Villareal de Garza. (2007). Introducción a la Computación, Segunda edición. México D.F. McGraw-Hill Interamericana.
- Luis Joyanes Aguilar. (2008). Fundamentos de Programación, Cuarta edición. Madrid. McGraw-Hill Interamericana.
- Fco. Javier Ceballos Sierra. (2012). JAVA Curso de Programación, Cuarta edición. Madrid. Alfa Omega - RAMA.